

A Self-Correcting World Model for Urban Air-Quality Fields on Heterogeneous Low-Cost Sensor Networks

Melchior Maison¹

¹Canarin Project, Paris, France {firstname.lastname}@canarin.example

October 8, 2026

Abstract

Dense networks of low-cost air-quality sensors promise street-level pollution maps, but their readings are noisy, irregularly sited, and of uncertain provenance: a device may be broken, indoors, or simply mislocated. We argue that the pollution field over such a network is best treated as a partially observed *dynamical system* and modelled with a controller-in-the-loop *world model* in the sense of Ha and Schmidhuber. Our central design principle is to factor the problem by *update rate*: near-static context (building geometry, altitude) conditions the model; exogenous weather acts as the control input that drives the dynamics; the pollution field is fast latent state; and per-device properties—reliability, indoor/outdoor exposure, and position—are slow state that gates how each observation is assimilated. Because these factors evolve on separate timescales, effects that are otherwise confounded (a wind-driven plume versus a street-canyon pocket; a faulty sensor versus a real transient versus a mislocated one) become separable. The controller does not act on the environment—one cannot steer the air—but on the system’s own trust in each device, an epistemic action space trained in imagination. We present the architecture, a heterogeneous-device conditioning scheme covering two sensor classes—an ambient station with onboard GNSS, and a personal-exposure instrument with none, whose power source governs its mobility, its exposure and its duty cycle—and runnable components validated against live public data. On a synthetic advective–diurnal field, a graph-regularized linear baseline attains $53.9 \pm 4.5\%$ skill over persistence at a six-hour horizon, establishing a concrete bar that a first recurrent core, trained on this field, then clears at every horizon beyond one step—cutting six-hour error by 27% over that baseline, with the mid-horizon advantage also transferring to real CAMS PM_{2.5} measurements. In a fault-injection study the trust controller down-weights faulty devices and cuts field-reconstruction error by 13% toward an oracle that is told which devices have failed; mislocation, catchable only in dense neighbourhoods, is left to dedicated position state. Because each device reports several pollutants, we also show that the channel most useful for predicting particulate mass—humidity, via hygroscopic growth—is almost uncorrelated with it, so grouping channels by correlation is misleading, and that such covariates must condition each device’s own reading rather than join a shared latent. We release the design and prototypes as a basis for deployment on the Canarin network.

1 Introduction

Low-cost particulate and gas sensors have made it feasible to instrument a city at the block scale rather than with a handful of reference stations. The resulting data, however, are hard to use directly. Individual devices drift and fail; they are placed by volunteers on balconies, indoors, and against building walls; their reported coordinates may be off by tens of metres. And because the network is *open*—it accepts readings from any device that can report, from dedicated boards to phones to third-party hardware—nothing about a contributor’s calibration, siting, or location can be assumed. A naive interpolation of raw readings inherits every one of these faults, and treating each sensor independently discards the strong spatial and temporal structure that makes the data valuable in the first place.

We take the position that the quantity of interest—the ambient pollution *field*—is a spatiotemporal dynamical system driven by known exogenous forcing (wind, mixing-layer height, the diurnal cycle) and shaped by static urban form, and that the sensors are noisy, partially trustworthy observers of it.

This framing invites a *world model* (Ha and Schmidhuber, 2018): a learned latent encoder, a recurrent dynamics core that can be rolled forward under hypothetical forcing, and a controller that closes a feedback loop. The world-model literature was developed for embodied agents that act to maximize reward. Our setting has no such lever—Canarin cannot change the air—so we retarget the controller onto the one thing the system *can* act on: its own beliefs about which sensors to trust. This is an epistemic, active-inference-flavoured control problem (Friston, 2010), and it can be trained “in the dream” against the model’s own predictions without waiting for real failures.

Our contributions are: (i) an *update-rate factoring* of the problem that makes otherwise-confounded effects identifiable (§3); (ii) a concrete mapping of the encoder/dynamics/controller triple onto a graph encoder, a stochastic state-space core with weather as the action input, and a reliability controller, together with a query-conditioned neural-field decoder for virtual sensors (§4); (iii) a heterogeneous-device conditioning scheme and an identifiability procedure that disentangles *faulty*, *real-micro-feature*, *indoor*, and *mislocated* explanations of a disagreeing sensor (§4); and (iv) runnable system components validated against live public APIs, a multi-seed baseline study that quantifies the bar the full model must beat, a fault-injection study that empirically validates the trust controller, and a trained recurrent core—deterministic, probabilistic, and autoregressive—that clears that bar and whose predictive bands we calibrate (§5–§6.8). We are explicit about scope: the recurrent core is trained on synthetic and real CAMS data, not on Canarin’s own operational readings, which we cannot access from here.

2 Related Work

World models and latent dynamics. Ha and Schmidhuber (2018) introduced the vision-model / memory-RNN / controller decomposition; latent state-space variants such as PlaNet (Hafner et al., 2019) and Dreamer (Hafner et al., 2020) showed that a stochastic recurrent core supports planning and control entirely in latent space. We borrow this decomposition but replace pixel frames with an irregular sensor graph and replace agent actions with meteorological forcing.

Spatiotemporal forecasting. Graph neural networks with recurrent or diffusion structure are standard for sensor-network forecasting (Kipf and Welling, 2017; Li et al., 2018). Such models typically assume trustworthy, correctly located nodes; our contribution is orthogonal and complementary: a mechanism for deciding *how much each node should count*.

Continuous decoders. Coordinate-based neural fields (Mildenhall et al., 2020; Sitzmann et al., 2020) map a location to a value, enabling queries at arbitrary points. We use a field decoder to read the latent state at unmonitored locations (“virtual sensors”).

Data assimilation. Sequentially correcting a dynamical state against noisy observations with an uncertainty budget is classical data assimilation; the ensemble Kalman filter (Evensen, 2003) is the canonical example. Our recurrent posterior update plays the same role, with the controller adjusting the observation-error covariance per device.

Low-cost air quality. The promise and pitfalls of low-cost sensing are well documented (Lewis and Edwards, 2016; Zheng et al., 2013); calibration drift and siting are the dominant error sources, motivating our explicit per-device reliability and exposure state. Urban canopy parameters such as street-canyon aspect ratio and sky-view factor are standard descriptors of dispersion geometry (Oke, 1988).

3 Problem Setting and Design Principles

A Canarin device is not a single-species instrument, and its channels are not all of the same kind. A Pico carries a Plantower PMSA003 particulate counter (three size bins), a Sensirion SGP40 gas sensor (a VOC index plus derived and raw signals), and Bosch BME280 and BME680 environmental sensors—so a single unit reports a dozen or so distinct `normalized_codes`, of at least four different semantic types (Table 2). Throughout the paper `normalized_code` is the canonical pollutant identity; in the deployed platform it is `WebFront.pollutant_types.id`, reached through each device’s channel declaration in

`WebFront.pollutants` (§5.1). Treating them as interchangeable “pollutants” would be a modelling error: only some are ambient field quantities at all.

Three distinctions are load-bearing. First, *a value’s role is not its units*. The particulate bins are ambient concentrations to be mapped and forecast; the VOC index is a bounded ordinal index, not a concentration; raw H_2 /ethanol signals and gas resistance are instrument-internal diagnostics; and pressure and humidity are ambient covariates that condition the interpretation of the others—optical particulate counts inflate at high humidity through hygroscopic growth. Second, and easy to get wrong: **the temperature channel measures the device’s own heat, not ambient air temperature**. It is a housekeeping and calibration covariate; the device cannot sense ambient temperature at all, which is precisely why the platform obtains it from the external weather service (§5) rather than from the fleet. Third, some channels are *duplicated within one device*—BME280 and BME680 both report pressure and humidity—which the `instance_index` disambiguates and which, as §6.12 notes, provides a free intra-device consistency check.

Four blocks, one of them open. The model reads from Plumage, which stores every observation as a tuple

$$(\text{device_id}, \text{instance_index}, \text{normalized_code}, \text{ts}, \text{value}), \quad (1)$$

so a record is recovered by grouping on `(device_id, ts)`. The codes in a record do not form one homogeneous vector: they fall into four blocks whose *arity stability* differs, and that difference—not the physics—is what should drive the architecture (Table 1). Three blocks have fixed, known width: GEO (latitude, longitude, and a quality covariate), HARDWARE (battery state and board temperature), and WEATHER (delivered by the weather service, which shares the normalized-code vocabulary but is keyed by grid cell and hour rather than by device—see below). Only the fourth, SENSING—pollutants and whatever else a contributor reports—is an open set that grows over the life of the deployment.

Two consequences follow, and they simplify rather than complicate the design. First, *only SENSING needs dynamic handling*; the other three use ordinary fixed-width encoders, so the complexity is confined to one block instead of infecting the whole input path. Second, the split lines up with the update-rate factoring of Table 3: GEO is near-static for a Pico and fast for a phone-powered Femto, HARDWARE is slow, WEATHER is the exogenous fast input u_t , and SENSING is the fast state being modelled. The blocks are therefore not an implementation convenience but the same seam the model is already built around.

A constraint attaches to HARDWARE: it must reach the trust controller and the observation model, never the field decoder. Battery level and board temperature are evidence about the *instrument*—a sagging battery precedes brownout, a hot board biases the collocated humidity and gas channels—and letting them predict concentrations would let the model learn “this hardware reads high,” the identical leakage the paper forbids for device type.

The types table carries the block. Block membership is a property of a *code*, not something the model should infer from the code’s spelling. Murmuration therefore keeps a type registry (`Murmuration.type_registry`, keyed by pollutant type) with the block and the finer role (`block` $\in$ {geo, hardware, weather, sensing}; `role` $\in$ {target, covariate, diagnostic, derived, housekeeping, exogenous}), together with the flags the model needs: whether a code gets a decoder head, whether it may join a tranche, its cold-start trust prior, its ingestion sanity window, and—for nested quantities—a `parent_code` and constraint kind. Enrolling a new sensor is then a row insert, not a code change, and the model derives its whole plan for a cycle from that table: which codes to decode, which to condition on, which pairs to gate, which are eligible for tranche assignment. The consistency gates in particular fall out of the data rather than being hardcoded—declaring `pm1` $\rightarrow$ `pm25` $\rightarrow$ `pm10` with `le_parent` is what generates the checks of §6.12. An unregistered code degrades safely: it lands in SENSING, receives no decoder head and a low trust prior, and contributes nothing until it is described.

Weather is stored once per cell-hour, not once per device. An earlier version of this design wrote the resolved weather back under each device’s own key. That is simple to read but wrong in one important respect: the model maintains a position correction Δp , so a device’s location can be revised after the fact, at which point per-device weather rows are attached to the wrong cell and must all be rewritten. We instead store weather once per `(cell_id, ts_hour)` and stamp each reading with the `cell_id` that was resolved for it—one integer per record rather than a set of float rows. This keeps the read-side join a plain key lookup, cuts weather storage by a further factor of twenty on the fleet

Block	Contents	Arity	Role
GEO	lat, lon (+ fix quality)	fixed	spatial weighting, position state
HARDWARE	battery, board temperature	fixed	health/trust; <i>never</i> the field
WEATHER	from weather service, keyed (cell, hour)	fixed	control input u_t
SENSING	pollutants and others	variable	fast state $\rightarrow$ tranches

Table 1: The four blocks of a record, joined by (`device_id`, `ts`). Only SENSING has variable width, so only it requires dynamic handling; the split coincides with the update-rate tiers of Table 3.

Sensor	Channels	Semantic type	Role in the model
Plantower PMSA003	PM ₁ , PM _{2.5} , PM ₁₀	mass concentration	ambient target
Sensirion SGP40	VOC index	bounded index (0–500)	ambient target (ordinal)
	eCO ₂	<i>derived</i> estimate	derived, not measured
	rawH ₂ , rawEthanol	raw sensor signal	diagnostic / calibration
BME680	gas resistance	raw sensor signal	diagnostic / calibration
BME280, BME680	pressure, humidity	ambient physical	covariate (duplicated)
BME680	temperature	<i>device self-heat</i>	housekeeping , not ambient
Weather service	ambient temp., wind, ...	external, per cell-hour	control input u_t

Table 2: Channels reported by a Canarin Pico, by semantic type. Only the first group are ambient quantities to be mapped; the temperature channel reads the device’s own heat and is never treated as ambient air temperature, which the platform takes from the weather service instead. Pressure and humidity appear twice per device, on two different sensors.

of Table 5, lets a position correction re-resolve cleanly, and—because the cell is recorded rather than recomputed—preserves the provenance needed to answer what the model actually saw, even if the grid resolution or provider later changes.

Tranches within the open block. Because SENSING alone varies, only it is split into *tranches*: small cores each covering a few codes, spawned and retired by a registry as codes appear and disappear, versioned and rolled back independently, and combined by fusing their *residuals* rather than their outputs (§6.12). Membership is decided by measured *predictive gain*—does one code’s history improve the forecast of another beyond its own history—and emphatically not by correlation, which we find points the wrong way (§6.12). A new code enters as a singleton tranche with near-zero trust, and is merged once enough history exists to measure whether it belongs anywhere, which is also when its credibility score (§6.11) first becomes meaningful.

We seek a latent field state z_t whose decoder $g(\cdot)$ predicts, at *any* location, each channel designated as an ambient target; a short-horizon rollout $z_{t+1:t+h}$ under a weather forecast; calibrated uncertainty; and a per-device trust weight. Trust is per *channel* as well as per device: a unit whose particulate counter has failed may still report humidity correctly, and amputating the whole device would discard good data.

Factor by update rate. The influences on $y_{i,t,c}$ change on very different timescales, and building the model around that seam is what makes them separable (Table 3). Corridors and heat pockets *emerge* from the interaction of a fast term (wind) and a slow term (geometry). Because wind varies while geometry does not, the two can be disentangled from observations alone: geometry is invariant across wind conditions, whereas the wind effect co-varies with wind. The same logic later separates a faulty sensor (a persistent signature) from a real transient (co-varying with conditions) and from a mislocated sensor (a residual explained by a spatial shift).

4 Model

We instantiate the world-model triple as an encoder V , a dynamics core M , and a controller C , with a separate neural-field decoder g (Fig. 1).

Tier	Quantity	Update rate	Role
Static context	altitude, canyon geometry, land use	$\sim$ never	conditioning
Exogenous	wind, temperature, mixing height	every step	control input u_t
Fast state	pollution field latent z_t	every step	evolved, assimilated
Slow state	reliability, exposure, position	hours–days	gate observations
Attribute	device type	fixed	selects obs./location model

Table 3: Update-rate factoring of the problem. Separating quantities by how fast they change is the load-bearing design choice: it renders confounded effects identifiable.

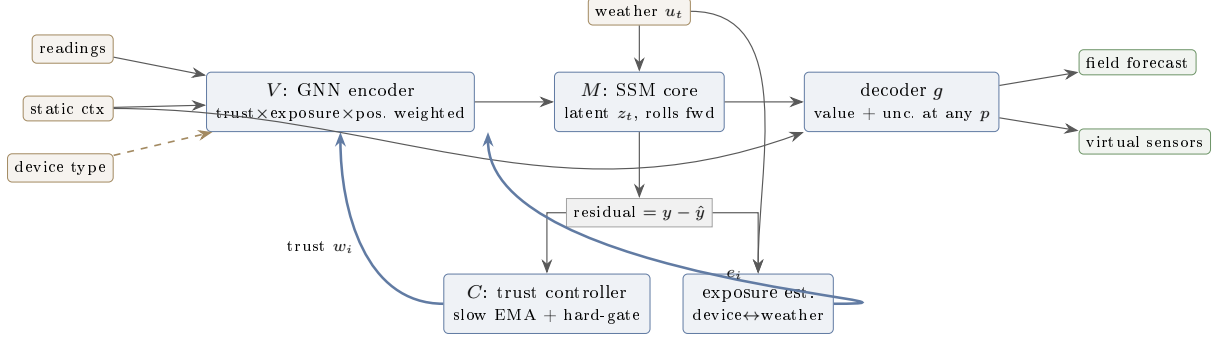


Figure 1: Architecture. Solid arrows carry data and feedback; the dashed arrow is *conditioning*: device type parameterizes the observation/location model rather than supplying a field value. Weather enters M as the control input u_t . The controller C and the exposure estimator produce slow per-device state that re-weights assimilation in V , closing the loop.

Encoder V . A graph neural network over the irregular sensor graph maps the current observation batch, static context, and weather to the field latent z_t . Each reading enters with weight proportional to $w_i \cdot e_i \cdot c_i$ —trust, exposure, and position-confidence—so that broken, indoor, or poorly located devices influence the field less without being discarded.

Dynamics core M . A stochastic state-space core (an RSSM-lite recurrence) propagates $z_t \rightarrow z_{t+1}$ with the weather forecast occupying the *action* slot of the classic memory RNN. Rolling the core forward under a forecast yields the multi-step field prediction; assimilating the next real batch is the posterior update of the state-space filter and supplies uncertainty bands, whose realized calibration we probe in §6.7.

Decoder g . A coordinate-conditioned neural field reads z_t at any (lat, lon) , producing both real-sensor reconstructions and virtual-sensor estimates with uncertainty. Keeping encode and decode separate is what enables queries away from device locations.

Controller C . Because there is no environmental action, C acts on the observation model itself: it maintains a per-device trust weight w_i that enters V ’s assimilation gain. Trust evolves as a slow, asymmetric exponential moving average of agreement,

$$w_i \leftarrow w_i + \eta (a_{i,t} - w_i), \quad \eta \text{ small}, \quad \eta^- > \eta^+, \quad (2)$$

where $a_{i,t}$ is the agreement between the device and its expected reading, and the asymmetry makes trust easier to lose than to regain. An unambiguous fault (flatline, dropout, rail-pinned value) bypasses the EMA through a hard gate, while a periodic *forced-listening* probe re-admits a distrusted device at full weight to test whether it has recovered. Trust can be trained against the model’s own imagined rollouts, so failures need not be observed in the wild first.

Multiple channels per device. Because each device reports a dozen typed channels, the encoder consumes the *set* $\mathcal{O}_t$ of Eq. (1) through per-code embeddings and per-code standardization—channels differ by orders of magnitude and in kind, so a single scaler is meaningless across a VOC index and a pressure in hPa. Decoder heads are emitted only for channels designated as ambient targets. Three

	Pico <i>ambient station</i>	Femto <i>personal medical instrument</i>
Hardware	LilyGO-class board + u-blox GNSS	own MCU, no GNSS
Power source	mains or battery	host phone, or mains
Connectivity	cellular or Wi-Fi (UDP)	via host phone, or own Wi-Fi
Location source	onboard GNSS	phone GPS, Wi-Fi positioning, or declared
Indoor fix	no (blind)	yes (non-GNSS)
Position error	multipath, sidereal-repeatable	tens to hundreds of metres
Quality covariate	HDOP, satellite count	position_source , accuracy radius
Duty cycle	continuous	gated by its power source
What it samples	the ambient field	the carrier's exposure
Role in assimilation	full weight	reduced or none when phone-powered

Table 4: Device archetypes. The classes differ in kind, not merely in error regime. A Pico is an ambient station; a Femto is a medical instrument measuring a person’s exposure, with no satellite receiver in either mode. Its power source is the governing variable—phone-powered implies carried, mobile and personal; mains-powered implies plugged in, static and almost certainly indoors—and connectivity, location and duty cycle all follow from it. Device type therefore selects the position prior, the exposure prior *and* the assimilation weight, without ever acting as a feature of the field itself. The fleet remains *open*: any device able to emit the wire format may contribute, so these are archetypes rather than an exhaustive list.

design points follow. (i) *Cross-channel coupling is real signal*: the three particulate bins are nested size fractions and move together, humidity drives apparent particulate mass through hygroscopic growth, and the VOC index responds to the same combustion sources as the fine fraction. A model that sees the set can use one channel to predict another. (ii) *Cross-channel physics gives density-independent fault gates*. Two hold on a single device with no neighbour at all: the nested-fraction inequality $PM_1 \leq PM_{2.5} \leq PM_{10}$, which the Plantower’s three bins make checkable directly; and the *duplicate-sensor* agreement between the BME280 and BME680 pressure and humidity channels on the same board, which should track within tolerance and whose divergence localizes the failure to one of the two sensors. This matters precisely where the trust loop is weakest (§6.3): spatial disagreement needs neighbours and collapses on sparse networks, whereas these gates read one device’s own channels. (iii) *Role, not units, decides use*: pressure and humidity condition the observation model; raw H_2 , raw ethanol and gas resistance feed calibration and health, not the field; eCO_2 is a derived estimate rather than a measurement and is never treated as ground truth; and the temperature channel, being device self-heat, is a health covariate whose *rise* is itself a fault signature—a hot board biases the collocated humidity and gas readings.

Sharing capacity across channels. Whether one core should model all channels jointly is an empirical question, not a matter of taste, and §6.12 answers it on the device’s real channels: every way of merging them into one flat state degrades the targets, so the core is factored into tranches over targets while covariates enter through per-device conditioning. The block decomposition of §3 is what keeps that arrangement stable as new codes arrive.

Per-device slow state and conditioning. Three properties are learned per device and one is declared. *Reliability* answers whether the device works; *exposure* answers whether it samples the outdoor field at all, learned as the strength of the device–weather coupling and best represented as a continuous coefficient that doubles as the assimilation weight; *position* is treated uncertainty-first—a shaky fix informs the field over a blurred location before any correction—with an optional, tightly-constrained Δp correction that exploits the decoder’s differentiability in query location. *Device type* is a known attribute that parameterizes the observation and location-error model and seeds cold-start priors (Table 4); crucially it conditions the *interpretation* of a reading, never the field value directly, so the model cannot learn a spurious “type X reads higher” shortcut.

Power source is the governing covariate, and exposure is not a fault. Two consequences follow from a device whose operation is gated by what powers it. First, *absence is not unreliability*. A phone-powered Femto stops when the phone is off, out of range or unplugged; the resulting gaps are duty cycle, not failure, and the trust controller must not penalize them. The credibility score counts effective fleet size by timeliness rather than enrolment (§6.11), which is correct here for a benign reason: such a device

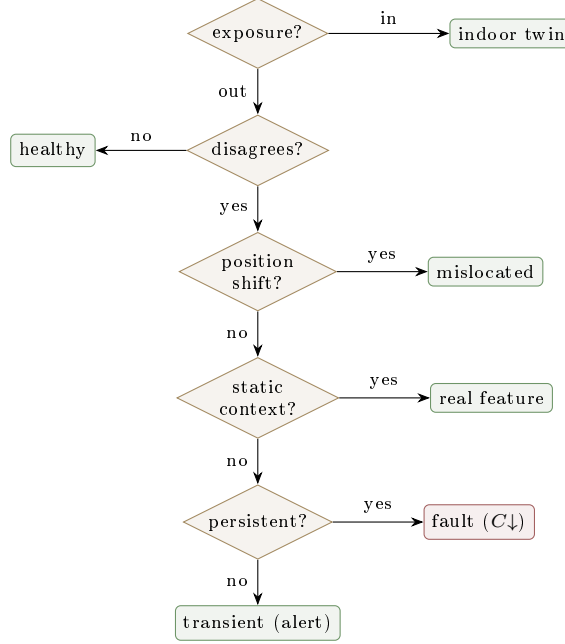


Figure 2: Resolving a disagreeing sensor. Static context, exposure, and position each absorb one benign explanation; only what survives all three is treated as a fault, and only a non-persistent survivor becomes an alert.

genuinely supports the live field only part of the time, while remaining sound.

Second, and more consequential for the architecture, *low outdoor coupling is the product rather than a defect*. Elsewhere in this paper a device decoupled from the ambient field is an *indoor fault* to be down-weighted (§6.3). For a Femto measuring a patient’s exposure it is the measurement. A mains-powered Femto is plugged into a wall socket and therefore almost certainly indoors; a phone-powered one follows its carrier through vehicles, offices and other people’s homes. Neither is evidence about outdoor air at its reported coordinates. The system therefore serves two products from one pipeline: Pico readings assimilate into the ambient field at full weight, while Femto readings carry the personal-exposure product at full value and enter the ambient field at reduced weight or not at all. The exposure estimator is seeded from the declared power source instead of having to infer it from residuals over days.

Reliability across the device hierarchy. Reliability need not be estimated per instance in isolation. Canarin devices form a natural hierarchy—*type* (Pico / Femto) over *device model* (a specific board or sensor revision) over *instance*—and reliability, together with the location-error scale, is partially pooled up this hierarchy: each instance’s trust prior is drawn from its device model’s distribution, and each device model’s from its type’s. Three consequences follow. (i) *Cold start*: a freshly enrolled unit inherits its device model’s prior rather than a flat one, so a new instance of a known-good model is trusted sooner and one of a known-flaky model is treated with caution. (ii) *Systematic-defect detection*: a fault shared by most instances of a device model shifts the *device-model-level* parameter, which is distinguishable from an instance-level failure—the difference between “this unit is broken” and “this revision is biased.” (iii) *Procurement signal*: the device-model-level posterior ranks revisions by field reliability, informing which hardware to deploy. This is a standard partial-pooling (random-effects) structure over the per-device trust state; it is specified here and left for training on operational data.

Identifiability: which twin? A device’s *twin* is the model’s expectation of its reading; exposure selects which twin it is judged against. A disagreement with the outdoor-field twin then has four possible causes, resolved in sequence (Fig. 2): a small position shift (mislocated), the static context (a real micro-feature such as a canyon pocket), temporal persistence (a fault), or none of these (a genuine transient, and an alert candidate). Reliability and exposure are orthogonal axes, so the system never “repairs” a healthy indoor sensor.

Inference loop. Each cycle (Algorithm-style summary): (1) read and normalize the batch; (2) encode with weights $w_i e_i c_i$ and the type-selected observation model; (3) roll M forward under the weather forecast and decode to real and virtual points; (4) assimilate the next batch, forming residuals against each twin; (5) update the slow state—trust via Eq. (2), exposure from device–weather coupling, position under its constraints; (6) write field forecasts, uncertainty bands, virtual sensors, anomaly scores, trust, exposure, and position to the store.

5 System Implementation

The model is not a standalone research artifact but a service embedded in the operational Canarin stack on AWS (one ECS Fargate cluster, an RDS MySQL 8.4 instance holding the schemas `Data`, `Devices`, `WebFront` and `Users`, and an ElastiCache Redis node). In that stack, ingestion is handled by *Birdhouse*, the authoritative store and dashboard by *Plumage*, alerting by *Chirping*, and ambient weather by *Thermal*; the world model runs as a separate service we call *Murmuration*. We describe the ingestion protocol and data model the model inherits, the two feature services, and—in detail—how Murmuration reads from and writes to the platform stores. This section describes the platform as deployed (October 2026, Murmuration v0.1); where the design goes beyond what is deployed we say so.

5.1 Ingestion protocol and data model (Birdhouse)

Canarin devices connect over cellular *or* Wi-Fi and typically sit behind carrier-grade or home NAT; they cannot be addressed inbound and therefore initiate every connection outbound, with bidirectional control over a cellular link requiring a private APN. Telemetry is emitted as compact **UDP** datagrams, which suit lossy, bandwidth-constrained links and keep the device firmware trivial. The wire protocol is deliberately *open*: any device able to emit a normalized datagram—a dedicated Canarin board, a smartphone, or third-party hardware—can contribute to the collection, which is exactly why the platform assumes nothing about a contributor and estimates trust, exposure, and position per device. The ingestion tier scales horizontally without a coordinator: multiple socket workers share one port via `SO_REUSEPORT` behind a network load balancer on ECS Fargate, so throughput grows with worker count.

Two invariants are established *at ingestion*, before any storage, and the rest of the platform—the model included—relies on them:

- **Unit normalization.** Each raw channel value is scaled by a per-pollutant `scale_factor` into canonical units, so magnitudes are comparable across device revisions and sensor vendors.
- **Identity normalization.** Each value is resolved to a *channel declaration*: a row of `WebFront.pollutants` keyed by `(device_id, pollutant_type_id, granularity, instance_index)` that names the canonical pollutant type (`WebFront.pollutant_types`: PM1, PM2.5, PM10, TVOC, CO2, humidity, pressure, temperature, battery, ...), carries the `scale_factor`, and maps the legacy per-device `value_id` the firmware still emits. Channels without a declaration are quarantined (`Data.data_points_quarantine`) rather than stored under a guessed identity—the “unregistered code degrades safely” rule of §3 is enforced one step earlier than the model. A composite primary key on `(device_id, time_bucket, pollutant_id)` in `Data.data_points_canonical` makes each observation a single, idempotent row; the timestamp is bucketed to the channel’s granularity.

The hot path performs only enrichment—an in-process, MySQL-backed cache of the channel declarations (five-minute TTL)—then does two things: it writes the canonical record to the authoritative store and *fans it out* by appending one entry to a Redis stream (`birdhouse:fanout`) on the shared ElastiCache node. The entry carries the device, the channel, the bucketed event time, the canonical value, and the server’s arrival time; the append is bounded in length and any Redis error is logged and swallowed. **Nothing model-related runs on this path**; Murmuration reads the stream with its own consumer group, strictly downstream, so inference latency can never back-pressure ingestion, and a consumer that is down simply resynchronizes from the store when it returns.

5.2 Feature services: weather and static context

Thermal is the design for the weather feature service; in the deployed platform ambient weather is fetched by Plumage’s *Ambient* service (Open-Meteo or Météo-France providers) for display, and it is not yet wired

into Murmuration as the control input u_t —v0.1 runs the kriging encoder of §5.3 without exogenous forcing. The rest of this subsection describes the service as designed and prototyped. Thermal resolves ambient weather per site from a public provider, decomposes wind into (u, v) components—interpolation on a direction *angle* across the $0/360^\circ$ wrap is incorrect—and interpolates the hourly forecast to the nowcast cadence; on provider failure it serves the last good value flagged stale and the model widens its uncertainty. Ambient weather is kept strictly distinct from a device’s *internal* temperature, which is a calibration and health covariate and, on Canarin hardware, the only temperature the device itself can report.

Attaching weather to every transmission. The weather block is defined per record, so Thermal enriches *each transmission*: it takes the reporting device’s position and timestamp, resolves the ambient conditions there and then, and writes them back into Plumage under the same composite key as the sensor rows. The read path is then a plain join on $(\text{device_id}, \text{ts})$ with no special-casing, which is what makes WEATHER an ordinary fixed-width block rather than a bespoke pathway.

Done naively—one provider request per transmission, one stored row per transmission—this does not survive contact with an open fleet, for three reasons, each with a corresponding measure (Table 5). *Volume*: thirty devices at twelve transmissions an hour is 60,480 requests a week, growing linearly with the fleet, and nearly all of them ask for the same number, because weather products are gridded and hourly. Quantizing $(\text{lat}, \text{lon}, t)$ to a $(\text{grid cell}, \text{hour})$ key and caching the result reduces this to one request per distinct cell-hour: for a Paris fleet spanning nine cells, 1,512 requests, a 97.5% reduction, and the fleet can grow without the provider load following it. *Hot-path coupling*: an outbound request inside ingestion would tie Birdhouse’s latency to a third party, violating the separation of §5.1, so the enrichment worker consumes the asynchronous fan-out instead; a provider stall delays enrichment and never ingestion. *Write amplification*: storing an hourly value once per transmission writes it a dozen times per device per hour. Writing on the hourly grid instead makes the upsert idempotent under the existing composite constraint and costs 91.7% fewer rows, losing nothing, since sub-hourly interpolation is deterministic and is applied at read time.

The fleet is also a barometer network. Pressure differs from every other conditioning channel in one respect: it is a genuine measurement of the ambient atmosphere, so the fleet does not merely consume the weather field but *observes* it. This is exactly the contrast with the temperature channel, which reads the device’s own heat and can only ever be hardware health. Three measurements determine how the observation can be used.

Individually the devices are blind, collectively they are not. After reduction to sea level the real between-site signal is 0.108 hPa, below a BME280’s own ± 0.12 hPa precision, so no single device can resolve the spatial field. Averaging reduces noise as $N^{-1/2}$: the fleet crosses into resolving it between four and sixteen devices, reaching 0.030 hPa at sixteen and 0.022 at thirty. Pressure assimilation is therefore a fleet-level product and never a per-device one.

The usable quantity is the tendency, and it needs no calibration. Raw pressure is a poor model input—it is dominated by the per-device altitude constant and degraded PM_{2.5} forecasting by 2.2%—whereas the three-hour change has inter-site correlation 0.962 and a signal-to-noise ratio of 6.6 against sensor noise. Because each device’s altitude offset is constant, it cancels exactly in the difference, so a device contributes tendency with neither a known elevation nor an absolute calibration. That matters in an open fleet, where mounting height—a sixth-floor balcony is some 18 m, about 2 hPa—is rarely recorded.

The cross-check runs both ways. One device disagreeing with the provider is a device fault; the whole fleet disagreeing, by more than several times its own standard error, is evidence that the *provider* is stale or mis-analysed—a condition no per-device trust loop can see, because every device appears equally wrong. With thirty devices the fleet’s standard error is 0.030 hPa, so it audits the provider to about 0.12 hPa: in simulation a 0.2 hPa provider error is flagged in every hour, while a correct provider is never flagged. The same consensus supports a device-local relocation check—a step in the altitude implied by a device’s barometer against the fleet mean identified a 60 m relocation as 55 m with no other device flagged—which is the mislocation mode the trust loop misses on sparse networks. Slow barometer drift is not caught by that step test, so we add a trend test on the same series. Monotonicity alone is insufficient—a step also trends—so the discriminator fits both a line and a step to the implied-elevation series and lets the better fit decide. On injected faults this separates the two cleanly: a 2 hPa ramp is flagged as drift and not as relocation, a 30 m relocation is flagged as a move with the step size recovered as 28.7 m, and clean data raises neither flag.

	naive	with cell-hour keying	reduction
provider requests / week	60,480	1,512	97.5%
weather rows / week	362,880	30,240	91.7%

Table 5: Cost of attaching weather to every transmission for a 30-device fleet at 12 transmissions per hour over one week, spanning nine grid cells. Quantizing to (cell, hour) and writing on the hourly grid leaves the read-side join unchanged while removing almost all of the provider and storage load.

Grid granularity is a per-variable question, and pressure is not a weather variable. The cell size should match the *source product*, since resolving finer than the product returns interpolated values that carry no additional information. Meteorological products over France resolve wind and temperature at ~ 1.3 km, whereas boundary-layer height is far coarser—so a single global cell size is either wasteful or lossy. Two measurements settle the choice. First, at 10 km separation the between-site difference is 24% of the temporal variability for $\text{PM}_{2.5}$ and 31% for humidity, so those fields do carry structure a coarse grid discards. Second, and more useful, *pressure does not*: across 16 Paris sites, terrain elevation explains 99.96% of the between-site variance in mean surface pressure (fitted slope -0.115 hPa/m against the barometric -0.12 , $r = -0.9998$), and correcting for altitude collapses the spread from 5.37 to 0.108 hPa. A fine pressure grid would spend requests re-learning a constant; one coarse cell plus a per-device elevation is strictly better.

That correction pays a second dividend. The altitude *implied* by the gap between a device’s own barometer and the sea-level field is a device-local check needing no neighbour: a step change in implied elevation indicates the device has moved—the mislocation mode the trust loop was measured to miss on sparse networks (§6.3)—while a slow ramp indicates a drifting pressure sensor, and the two are separable by their time signature.

Whether per-variable grids are worth splitting requests over depends on the provider, and we found the intuitive answer to be wrong. Grouping the codes by resolution costs *more*, not less: on the fleet of Table 5, splitting into four resolution groups took 9,744 requests a week against 4,536 for a single bundled request per cell-hour at the finest required resolution. Because a typical provider returns many variables in one response, multiplying requests by the number of groups outweighs the cells saved on the coarse ones. Splitting only pays where requests are billed per variable. We therefore bundle by default and retain the split path for providers that charge that way.

Two behaviours matter operationally. On provider failure the worker serves the last good value for that cell within a TTL and writes an explicit `wx_stale` marker alongside it, so the model can widen its bands rather than consume a silently old number; past the TTL it writes nothing at all rather than guessing. And because the cache key is derived from *position*, a mobile Femto crossing cells correctly triggers a fresh resolution, while a static Pico reuses its cell indefinitely. A device that was offline and later flushes buffered records carries old timestamps; the hour is part of the key, so those resolve against historical conditions rather than present ones—provided the archive endpoint is used for hours outside the live window.

The static-context loader derives per-location geometry (plan-area index, mean height, canyon aspect-ratio and sky-view-factor proxies) from a public elevation API and OpenStreetMap footprints, imputes missing values from the fleet median, and distinguishes a *genuinely open* site from an *unknown* one so a dense-but-untagged block is never read as open sky. Both are first-order proxies; a rasterized surface model with ray-casting is the production upgrade.

5.3 Model–database interaction (Murmuration)

Murmuration touches four stores, each for a distinct reason (Table 6, Fig. 3). It never reads the raw device stream; it consumes the *normalized* vocabulary established by Birdhouse (§5.1), so no schema translation occurs anywhere between ingestion, storage, and the model.

Reads. Offline *training* pulls a historical corpus from the authoritative store as tuples of the form Eq. (1)—in deployment a join of `Data.data_points_canonical` to `WebFront.pollutants` that selects the pollutant type and instance—grouping them into a *time* $\times$ *device* $\times$ *code* structure whose code axis is discovered from the data rather than fixed in advance. Device position comes from `Devices.devices_hardware` (last fix, HDOP, satellite count); the declared attributes the platform does

not hold—device type, power source, hardware revision—live in `Murmuration.device_profile`, an optional row per device that seeds the priors of Table 4. Weather is registered in the *same* normalized-code vocabulary but stored under a different key—by `(cell_id, ts_hour)` rather than by device—so it is reached through the `weather_cell_id` stamped on each reading rather than by a device-keyed join. Sharing the vocabulary is what lets the type registry describe it like any other code, with its *role* as control input u_t recorded as metadata; not sharing the key is what keeps one copy per cell-hour instead of one per device. Trained weights are versioned to S3. Online *inference* consumes the newest canonical batch from the `birdhouse:fanout` stream—never the hot path—re-claiming any entries read but not acknowledged by a consumer that crashed mid-cycle. The fast mutable state (the field latent z_t) and the slow per-device state (trust w_i , exposure e_i , position) live in Redis/ElastiCache for low-latency read-modify-write across the ~ 5 -minute nowcast cycle, with authoritative copies persisted to Plumage. Weather is a keyed cache lookup in Thermal, not a live fetch.

Writes. Each cycle Murmuration upserts six result relations into the shared database—`field_forecast`, `virtual_sensor`, `anomaly_score`, `device_trust`, `device_exposure` and `device_position`—keyed by the same composite identity plus a `model_version`. The six are split across two schemas by a privacy rule that the schema itself enforces: a public prediction names a pollutant at a place and a time, never its source. `Murmuration.field_forecast` and `Murmuration.virtual_sensor` carry no device identifier and no contributor list; the four device-level relations live in `MurmurationInternal`, which Plumage exposes to administrators only. Every row additionally carries a `credibility` score in $[0, 1]$ and its band, computed from the fleet size and history length behind that output (§6.11), so the dashboard can render a provisional estimate differently from a well-supported one and Chirping can suppress alerts whose evidence is thin. Because the keys match the ingestion vocabulary, Plumage’s dashboard queries and Chirping’s alert rules join model outputs to raw observations with no translation; `device_trust` in particular is what lets Chirping separate a real pollution spike from a failing sensor.

Event time is not arrival time. The fan-out decouples ingestion from inference, which is what keeps a slow or failed consumer off the hot path; it also introduces a hazard on the consuming side. A device behind CGNAT that has been unreachable buffers its readings and flushes them on reconnection: those records *arrive* now and *belong* to hours ago. Birdhouse is untroubled by this—its writes are idempotent on the device-stamped `ts`, so a flushed record lands in the correct historical row—but a consumer that treats the newest batch as the present will assimilate stale observations into the live state, and the trust controller will then see a large residual and begin distrusting a device whose only fault was poor connectivity. The failure penalizes precisely the devices an open network most wants to retain.

We therefore route by event time. Records within the current cycle window are *fresh* and may update both the field state and trust. Records older than the window but inside a reanalysis horizon are *late*: they are written to the store and admitted to the training corpus, where only their timestamp matters, but they never move a live weight, and the hours they touch may be marked for bounded reanalysis. Older still is *stale*, stored without revisiting the field. Timestamps ahead of the present beyond a small tolerance are treated as device clock skew: stored, flagged, never assimilated.

The asymmetry—late data is good enough to train on but not to filter with—is the point, and the cost of getting it wrong is large. Replaying a device’s buffered readings as if current costs it 42.5% of its trust after a twelve-hour outage and 55.6% after a day; during a volatile period a six-hour outage alone costs 41.9%. Under the routing rule all of these are zero.

Cadence and consistency. A cycle reads a consistent z_t snapshot from Redis, assimilates the queued batch, and writes forecasts and updated state. All write-backs are idempotent upserts on the composite keys, so a retried or duplicated cycle cannot corrupt the store. The fast state turns over every cycle; the slow per-device latents evolve over hours to days and are therefore cached in Redis and checkpointed to the database periodically rather than rewritten each cycle. S3 artifacts are immutable and versioned, so a model rollback is a pointer change.

What is deployed (v0.1). Murmuration v0.1 (`canarins/canarin_murmuration`) implements this section with the *kriging encoder* in place of the learned core: the anomaly field is a Gaussian process with the exponential covariance of §6.9 whose per-device nugget is τ^2/w_i , so trust, exposure and position confidence enter exactly where §4 puts them, as the observation-error covariance; the anomaly is rolled

Interface	Store	Dir.	Payload	Cadence
Training corpus	RDS MySQL (Data , WebFront , Devices)	R	canonical readings joined to channel declarations + device position	offline retrain
Live batch	Redis stream birdhouse:fanout	R	newest canonical observations, with event and arrival time	~5 min
Field state	Redis	R/W	latent z_t	per cycle
Slow state	Redis $\rightarrow$ MySQL	R/W	trust, exposure, position	cycle; checkpoint
Weather [†]	MySQL	R	weather_cell rows via each reading's weather_cell_id	per cycle
Weather attach [†]	MySQL	W	weather_cell (cell, hour); weather_cell_id on readings	per transmission
Artifacts	S3	R/W	versioned model weights	on retrain
Outputs	MySQL (Murmuration , MurmurationInternal)	W	6 relations, keyed by identity + model_version , each with credibility	per cycle

Table 6: Murmuration’s interfaces to the Canarin stores (R=read, W=write). The model consumes and emits the same normalized vocabulary (**scale_factor** / pollutant type / **instance_index**) established at ingestion, so no schema translation occurs across the pipeline. [†]Designed, not yet deployed (§5.2).

forward as an AR(1) process with climatology, so predictive bands widen with horizon and are rescaled by the one-parameter recalibration of §6.8. The reliability controller, the device-local gates, hierarchical cold-start priors, event-time routing, credibility scoring and the six output relations are implemented as described. Kriging is retained permanently as the bootstrap and fallback encoder; the recurrent core of §6.5 replaces it behind the same interfaces once trained on operational data. On the synthetic fleet of §6.3, with a device dark for six hours and then flushing its buffer, the deployed service hard-gates the stuck sensor, drives the drifting one to the trust floor, leaves healthy sensors between 0.5 and 0.97, moves the dark device’s trust by at most one update step on the flush, and reaches +45% to +60% skill over persistence at one to six hours with 0.95 empirical coverage of its 90% bands. The service runs as one Fargate task on the existing cluster, holds its slow state in the shared ElastiCache node under its own key prefix, and is deployed by the same digest-pinned workflow as Birdhouse.

6 Experiments

We report twelve studies, in four groups. *Establishing the bar*: baseline forecasting on a synthetic field, and the calibration of that baseline’s uncertainty (§6). *Inputs and instruments*: live static-context extraction, fault injection against the trust controller, and hierarchical reliability across the device fleet. *Clearing the bar*: training the recurrent core, transferring it to real CAMS PM_{2.5}, adding a stochastic latent, rolling it out autoregressively, and comparing it against a derived ensemble Kalman filter. *Deployment questions*: how late-arriving data verifies the model for free, how output credibility depends on fleet size and history, and what several channels per device buy. Together they establish the design’s inputs, its bar, and a first trained core that clears it; training on Canarin’s own operational data remains future work.

6.1 Baseline forecasting

Data. We use a synthetic field with genuine spatiotemporal structure: a diurnal base with two rush-hour peaks, a source plume advected by a slowly rotating wind, a spatial gradient, and AR(1) sensor noise, over 12 sensors and 40 days at hourly cadence. We average over 30 independent realizations (seeds) and hold out the final 25% of each series for testing.

Methods and metric. We compare *persistence* ($\hat{y}_{t+h} = y_t$), *diurnal-persistence* (persistence plus a train-only hour-of-day climatology correction), and a *graph-lite* spatiotemporal model: a ridge readout over a device’s own recent anomalies and an inverse-distance neighbour-anomaly message, predicting the target-time climatology plus a learned correction. The graph-lite model is given the *same* diurnal information as diurnal-persistence, so the comparison isolates what spatial and temporal structure adds.

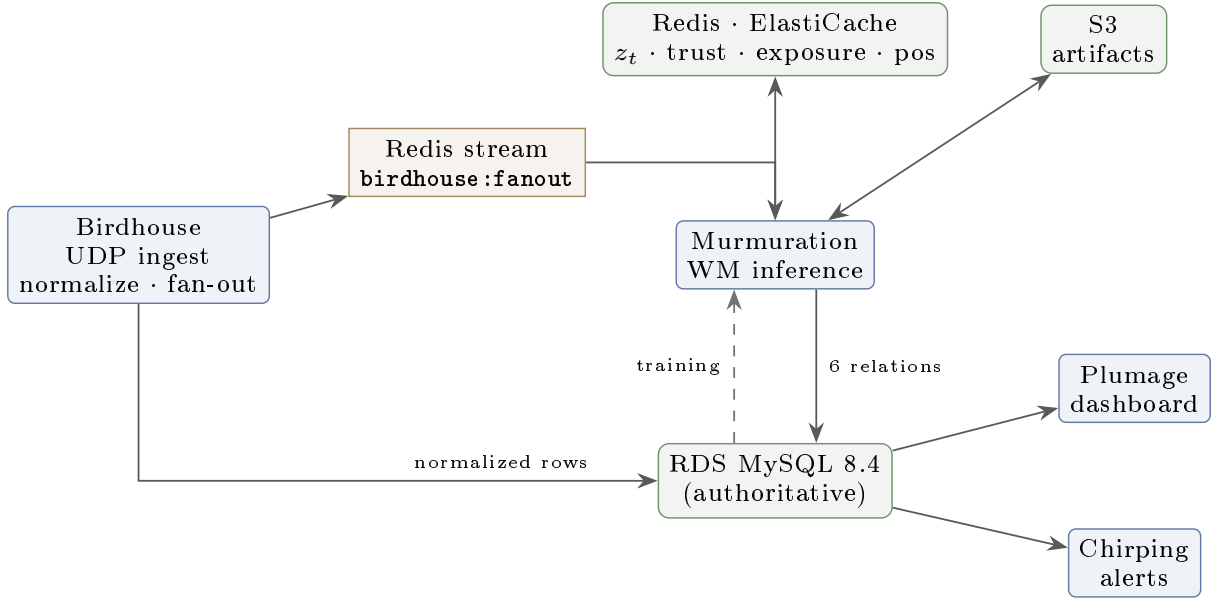


Figure 3: Data flow around Murmuration. Birdhouse writes canonical rows to the database and fans them out through a Redis stream; the model consumes the fan-out (off the hot path), holds fast state z_t and slow per-device state in Redis, loads versioned weights from S3, and upserts six result relations back to the database, where the dashboard and Chirping consume them. The dashed edge is the offline training read. Weather (Thermal) is omitted for clarity; see Table 6.

h (h)	RMSE (norm. units)		Skill vs persistence	
	diurnal-pers.	graph-lite	diurnal-pers.	graph-lite
1	0.272 ± 0.004	0.281 ± 0.005	$+27.4 \pm 3.8\%$	$+25.2 \pm 3.2\%$
2	0.350 ± 0.007	0.348 ± 0.006	$+42.1 \pm 4.5\%$	$+42.5 \pm 4.1\%$
3	0.409 ± 0.011	0.396 ± 0.009	$+48.5 \pm 4.6\%$	$+50.1 \pm 4.3\%$
6	0.515 ± 0.018	0.480 ± 0.015	$+50.5 \pm 5.0\%$	$+53.9 \pm 4.5\%$
persistence RMSE: 0.376/0.608/0.799/1.048 at $h = 1/2/3/6$.				

Table 7: Forecast skill on the synthetic field, mean $\pm$ std over 30 seeds. Graph-lite overtakes diurnal-persistence as the horizon grows, where spatial propagation matters.

Skill is $1 - \text{RMSE}_{\text{model}}/\text{RMSE}_{\text{persistence}}$. For the virtual-sensor task we hold out each sensor in turn and reconstruct it by inverse-distance weighting of the others.

Results. Table 7 and Fig. 4 report the results. Both structured baselines beat persistence by a wide and growing margin as the horizon lengthens—persistence collapses precisely because it ignores the diurnal cycle—reaching $53.9 \pm 4.5\%$ skill for graph-lite at $h = 6$. The graph-lite model overtakes pure diurnal-persistence only at longer horizons, where spatial propagation of the advected plume carries information the single-site climatology cannot. The leave-one-out virtual-sensor reconstruction attains $\text{RMSE} = 0.880 \pm 0.117$. **The operational lesson is that the recurrent core must be measured against the graph-lite row, not persistence:** a strong linear spatiotemporal model already captures roughly half the reducible error, and that is the bar a heavier model must clear to justify itself (§6.5 shows the trained core clears it at every horizon ≥ 2).

Calibration. The forecaster above is a point predictor, but the world model will emit uncertainty bands, so we establish the protocol and a baseline here. We equip graph-lite ($h = 3$) with a predictive variance and measure the empirical coverage of its central intervals against the nominal level (Table 8). A single train-residual variance (homoscedastic) is *underconfident*—its 90% interval covers 98%—while a heteroscedastic estimate from a log-variance regression is *overconfident* ($90\% \rightarrow 78\%$); neither is usable as-is (expected calibration error ≈ 0.10 – 0.12). A one-parameter recalibration—a single scale factor chosen on a validation slice—restores coverage almost exactly (error 0.013). This sets a bar: the recurrent core’s

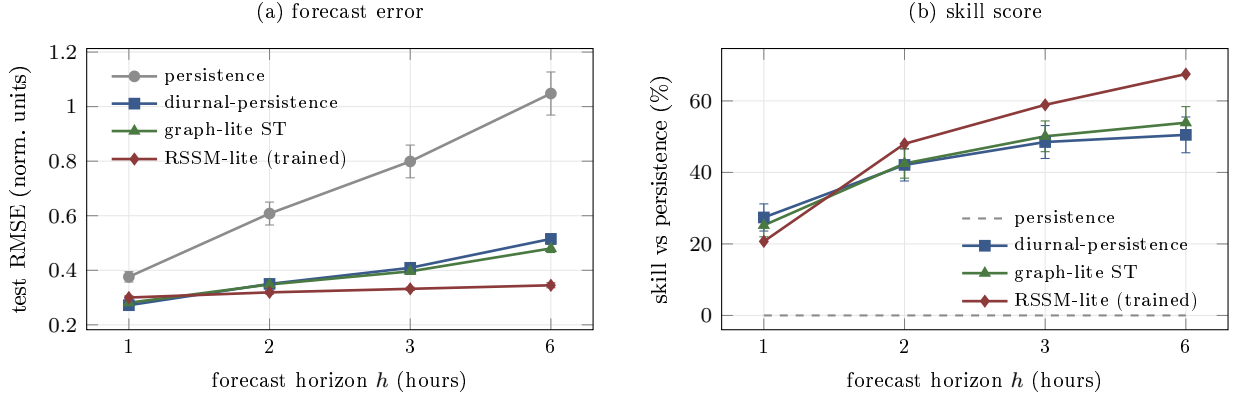


Figure 4: Forecast error (a) and skill over persistence (b) versus horizon, mean over 30 seeds with ± 1 std bars. The persistence baseline (dashed, right) is the zero-skill reference; the gap between it and graph-lite is what the recurrent core inherits, and the gap *above* graph-lite is what it must earn. The trained RSSM-lite core (red, 8 seeds) is overlaid: it falls below graph-lite at every horizon ≥ 2 , earning that gap.

nominal	homoscedastic	heteroscedastic	recalibrated
0.50	0.658	0.382	0.520
0.80	0.926	0.655	0.818
0.90	0.977	0.776	0.911
0.95	0.992	0.853	0.955
ECE	0.101	0.121	0.013

Table 8: Empirical coverage of graph-lite’s central prediction intervals versus the nominal level ($h = 3$, 20 seeds); ECE is the mean $|\text{empirical} - \text{nominal}|$. Naive bolt-on variances miscalibrate in both directions; a single validation-chosen scale factor fixes coverage.

assimilation posterior should be calibrated *without* such a post-hoc fudge, and this is the protocol by which we will check it.

6.2 Static-context extraction (live)

Table 9 shows features extracted live for three Paris sites of contrasting morphology. The dense boulevard site yields a high plan-area index, high aspect ratio, and low sky-view factor—a poorly ventilated street canyon—while the park-edge site is correctly resolved as open sky. One site’s building query timed out at the provider and fell back to fleet-median imputation, exercising the degradation path the pipeline is designed to handle.

6.3 Fault injection and the trust controller

To test the controller’s central claim—that it down-weights bad devices without discarding good ones—we inject the four fault modes of §4 into the synthetic field: a *stuck* flatline, a slow additive *drift*, an *indoor* device decoupled from the field (low exposure), and a *mislocated* device reporting the field at the wrong coordinates. On a 36-device network, six devices fail at $t = 10$ d and one drift device recovers at $t = 17.5$ d. The controller calibrates each device’s expected disagreement against its trust-weighted neighbours during a clean warm-up, then runs the asymmetric-EMA update of Eq. (2) with a flatline hard gate and a trust floor. We compare field reconstruction on a 16×16 grid under three weightings: uniform, trust-weighted, and an *oracle* that drops the faulty devices outright.

Figure 5 reports the outcome over 30 seeds. Trust of stuck and drifting devices collapses within hours to a day or two of onset, healthy devices stay near one (false-positive rate 8.6%), and the recovering drift device climbs back once its fault clears. Trust weighting cuts reconstruction RMSE from 0.179 (uniform) to 0.155, a 13% reduction that recovers most of the way to the oracle floor of 0.136. At the density simulated here the *mislocated* device is also caught—its neighbours are close enough to contradict a wrong-location reading—but this is the one result that does not survive sparsification (see below). The

Site (Paris)	elev. (m)	#bldg	λ_p	aspect	SVF
Dense boulevard	51	157	0.38	1.48	0.56
Park edge (open)	48	0	0.00	0.00	1.00
Riverside [†]	36	—	<i>imputed (provider timeout)</i>		

Table 9: Live static-context features. λ_p : plan-area index; SVF: sky-view-factor proxy. [†]Building query returned a gateway timeout; values imputed from the fleet median, as designed.

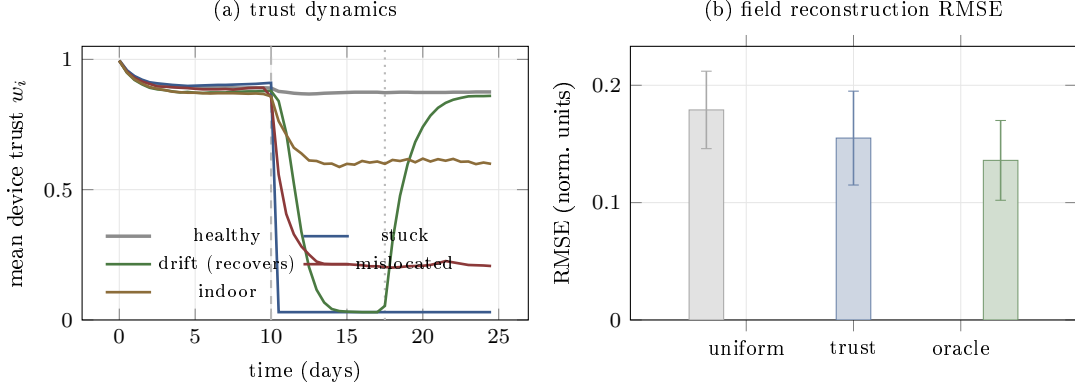


Figure 5: Trust controller under injected faults: 30 seeds, 36 devices, six faults from $t = 10$ d, with the drift sensor recovering at $t = 17.5$ d (dotted). (a) Category-mean trust. Healthy devices stay near one; stuck, drift and mislocated devices lose trust; the drift sensor recovers once its fault clears. Pure *indoor* (exposure) faults are slowest to catch by disagreement alone, motivating the separate exposure estimator. (b) Field-reconstruction RMSE. Trust weighting (0.155) improves on uniform (0.179, -13%) and approaches the oracle that drops faulty devices outright (0.136).

remaining hard case is the pure *indoor* fault: a device sitting at its baseline only disagrees when the field departs from that baseline, so it is caught slowly by disagreement alone. This is not a failure of the controller but evidence for the architecture—reliability and exposure are orthogonal axes, and an indoor device is an *exposure* problem for the exposure estimator, not a reliability fault for the trust loop.

A sparser, fully reproducible replication. Because the mislocation result is density-sensitive, we re-ran the study independently on a sparser network (16 devices, four faults, 25 seeds) with a released script. It reproduces the core findings: the flatline hard gate catches the stuck device in ~ 15 h, drift and indoor within two to three days, healthy devices are barely touched (mean final trust 0.85, false-positive rate 3.7%), and trust recovers about 89% of the fault-induced reconstruction error (a fault-free floor of 0.885, degraded to 1.329 without trust and restored to 0.934 with it). The one divergence is exactly mislocation: with neighbours further apart, a device reporting a plausible value from ~ 1.6 km away is caught in only 12% of seeds. Mislocation detection therefore scales with network density; below it, mislocation is a *position* problem for the position state, not a *reliability* problem for the trust loop—the empirical face of the identifiability argument in §4.

6.4 Reliability across the device hierarchy

Section 4 argued that reliability should be pooled across the type $\rightarrow$ device-model $\rightarrow$ instance hierarchy; we test that on a simulated fleet of six device models, eight instances each, one of which is a defective revision (a systematic low-reliability shift). Each instance yields noisy agreement measurements; half are *mature* (many observations) and half *cold* (few). We estimate each instance’s reliability three ways—no pooling (instance alone), complete pooling (one fleet value), and partial pooling (two-level empirical-Bayes shrinkage)—and score against ground truth (Table 10).

Partial pooling is best overall and decisively so where it matters: on cold instances it nearly halves the error of estimating each alone (0.27 vs 0.57), because a data-poor unit borrows strength from its device model. For a brand-new instance with no observations, its device-model posterior is a far better prior than the fleet mean (RMSE 0.29 vs 0.87)—the cold-start benefit the deployed system needs. The defective revision is correctly identified as the lowest-reliability model in 93% of runs, separating *this*

estimator	reliability RMSE (lower is better)		
	all	cold ($n=3$)	mature ($n=40$)
no pooling	0.418	0.570	0.156
complete pooling	0.872	0.869	0.874
partial pooling (EB)	0.235	0.269	0.187

Table 10: Per-instance reliability estimation on a simulated fleet (300 seeds). Partial pooling across the device hierarchy wins overall and on data-poor cold instances; for data-rich mature instances estimating each alone is marginally better, as hierarchical shrinkage predicts.

h (h)	test RMSE (norm. units)			
	persistence	graph-lite	RSSM-lite	Δ skill
1	0.378	0.280	0.300 ± 0.006	−5.4 pp
2	0.614	0.346	0.319 ± 0.006	+4.3 pp
3	0.809	0.394	0.332 ± 0.006	+7.6 pp
6	1.059	0.475	0.345 ± 0.009	+12.3 pp

Table 11: Trained RSSM-lite core versus baselines (8 seeds). Δ skill is the recurrent core’s skill-vs-persistence minus graph-lite’s, in percentage points. The core beats graph-lite at every horizon ≥ 2 by a widening margin; at $h = 1$ the linear baseline is marginally better.

revision is biased” from *“this unit is broken.”* As expected, for *mature* instances plain estimation is marginally better (0.16 vs 0.19): once a unit has enough data of its own, shrinkage adds a little bias, so the pooling correctly fades as observations accumulate.

6.5 Training the recurrent core

The baselines set a bar; here we clear it. We implement the dynamics core M as a deterministic-latent gated recurrent unit (an RSSM-lite core, without the stochastic latent) that encodes the whole sensor field’s history into a latent h_t and decodes it with a direct readout head per horizon. It is trained on the synthetic field by back-propagation through time—hand-derived and gradient-checked to 3×10^{-9} , as no autodiff framework was available—seeing only observations and hour-of-day, the *same* information as graph-lite, with the hidden wind forcing withheld to keep the comparison fair. Table 11 and Fig. 4 report the result over 8 seeds.

The recurrent core clears the graph-lite bar at every horizon beyond one step, and the margin widens with lead time: at $h = 6$ it cuts RMSE from 0.475 to 0.345, a 27% relative reduction, for +67.5% skill over persistence against graph-lite’s +55.2%. The mechanism is the one the design predicts—a recurrent latent captures the advective dynamics, so its forecasts degrade far more slowly than a memoryless linear readout. At $h = 1$ the cheap baseline stays marginally sharper (0.280 vs 0.300): a one-step nowcast has little dynamics to model, and the linear anomaly-plus-climatology readout is hard to beat there. This is the paper’s central premise with evidence on synthetic data: the recurrent core earns its place precisely where multi-step field evolution matters.

6.6 Beyond the synthetic field: real PM_{2.5}

A trained core that beats a baseline on the very field it was tuned against proves little unless the advantage transfers. We therefore repeat the forecasting comparison on data we did not design: hourly PM_{2.5} from the CAMS European air-quality analysis (via a public API) at 16 locations across greater Paris over 90 days. This is a modeled, gap-free, fault-free product—smooth and highly spatially correlated (mean inter-location correlation 0.94)—so it exercises the forecasting core, not the fault or reliability machinery, and it is not Canarin’s own Plumage stream, which we cannot access from here. The harness, split, and core are otherwise unchanged; the core is averaged over four initializations.

The mid-horizon advantage transfers (Table 12). The recurrent core clearly beats graph-lite at $h = 2$ and $h = 3$ (0.76 vs 0.83 and 0.98 vs 1.06 $\mu\text{g}/\text{m}^3$) and edges it at $h = 6$; at $h = 1$, real PM_{2.5} is so autocorrelated that plain persistence wins and every model sits within a few percent of it. The margins are smaller than on the synthetic field—unsurprisingly, as the CAMS product is spatially smooth, offering

h	test RMSE ($\mu\text{g}/\text{m}^3$)				skill vs pers.	
	persist.	diurnal	graph-lite	RSSM-lite	RSSM	g-lite
1	0.54	0.50	0.55	0.55	−2.4%	−2.1%
2	0.90	0.83	0.83	0.76	+15.4%	+8.0%
3	1.20	1.09	1.06	0.98	+17.9%	+11.5%
6	1.78	1.62	1.50	1.48	+16.8%	+15.8%

Table 12: Forecasting on real CAMS PM_{2.5} (16 locations across greater Paris, 90 days; RSSM over 4 inits). The recurrent core’s mid-horizon advantage over graph-lite transfers to real data; at $h = 1$ persistence dominates highly autocorrelated PM_{2.5}.

nominal	raw bands	recalibrated
0.50	0.379	0.537
0.80	0.648	0.829
0.90	0.766	0.915
0.95	0.841	0.957
ECE	0.129	0.030

Table 13: Calibration of the probabilistic core’s predictive intervals (synthetic field, 4 seeds, coverage averaged over horizons). The trained core’s raw bands are overconfident; a one-parameter per-horizon recalibration on a validation slice restores coverage, matching the protocol of §6.

less advective structure for a recurrent latent to exploit than the synthetic plume—but the ordering that matters holds on real data: the recurrent core earns its place at the horizons where field evolution, not mere persistence, drives the forecast.

6.7 A probabilistic core and its calibration

The design promises calibrated uncertainty from the core’s posterior; we test that directly. We give the recurrent core a stochastic latent—a reparameterized Gaussian $z_t \sim \mathcal{N}(\mu(h_t), \sigma(h_t)^2)$ regularized toward $\mathcal{N}(0, I)$ —and a heteroscedastic Gaussian decoder that outputs a mean *and* a variance per horizon, trained by an ELBO (negative log-likelihood + β KL; hand-derived gradients, checked to 4×10^{-9}). Predictive bands come from Monte-Carlo over the latent via the law of total variance. Point accuracy is unchanged from the deterministic core (RMSE 0.30/0.33/0.34/0.35 at $h = 1/2/3/6$), so probabilistic training costs nothing in sharpness.

Two honest findings follow (Table 13). First, on this mostly *observed* forecasting task the latent *collapses*: it supplies under 1% of the predictive variance. With direct per-horizon heads the heteroscedastic decoder already captures horizon-dependent spread, so the latent is redundant; realizing uncertainty that grows with horizon requires rolling the state forward autoregressively rather than reading each lead off a static head, which we do next (§6.8). Second, the decoder’s *raw* bands are overconfident—a 90% interval covers 77% (ECE 0.13)—because a variance fit to training residuals under-covers the larger errors on held-out data, the same scale gap the naive baselines showed in §6. A single per-horizon recalibration on a validation slice fixes it (ECE 0.13 $\rightarrow$ 0.03; the 90% interval 0.77 $\rightarrow$ 0.92). This corrects an over-optimistic reading of the design: the trained core does not emit calibrated bands “for free”; it emits well-*shaped*, input- and horizon-dependent bands whose *scale* still needs a one-parameter recalibration.

6.8 Autoregressive rollout: uncertainty that compounds

The direct-head core reads every horizon off one latent, so its bands cannot grow with lead time on their own and its latent sat idle. We therefore replace the four static heads with a single *one-step* heteroscedastic model— $o_{t+1} \sim \mathcal{N}(m(h_t), \exp lv(h_t))$, trained by one-step NLL (gradients checked to 2×10^{-7})—and forecast by *rolling it forward*: sample o_{t+1} , feed it back with the known next-hour features, advance the GRU, sample o_{t+2} , and so on. With $S = 60$ sampled trajectories per start, the ensemble spread at $t + k$ is the predictive band.

Now the bands *compound* (Table 14): the mean predictive standard deviation grows monotonically with horizon (0.245 $\rightarrow$ 0.321 in standardized units), because each step’s noise is carried forward through

h	predictive band		point RMSE	
	band std	raw 90% cov.	rollout	graph-lite
1	0.245	0.744	0.290	0.280
2	0.269	0.711	0.343	0.346
3	0.284	0.708	0.369	0.393
6	0.321	0.742	0.384	0.473

Table 14: Autoregressive rollout (synthetic field, 4 seeds, $S = 60$). The predictive band grows with horizon—uncertainty compounds through the recurrence—and raw 90% coverage is roughly uniform across horizons, so one global scale recalibrates all leads (ECE 0.166 $\rightarrow$ 0.013). Rolling out slightly worsens point accuracy versus the direct-head core but still beats graph-lite at $h \geq 2$.

the recurrence—exactly the behaviour a static head cannot produce. Two things follow. The raw bands are still overconfident (a one-step variance fit to training residuals under-covers), but *uniformly* so across horizons—raw 90% coverage is 0.71–0.74 at every lead—so a *single global* scale recalibrates all horizons at once, reaching ECE 0.013, better than the direct head’s 0.030 which needed a separate scale per horizon. The cost is honest: rolling out compounds error, so point accuracy slips relative to the direct head ($h = 6$ RMSE 0.38 vs 0.35) while still beating graph-lite (0.47). The rollout thus fixes the *shape* of the uncertainty; the remaining global offset is removed by a single validation-chosen scale, and—as we show next—that scale suffices even conditionally.

The global scale is enough, conditionally too. One might worry that a single scale merely trades over-coverage in calm periods against under-coverage in volatile ones. It does not. Splitting the test starts by local volatility (the mean absolute one-step change at the forecast origin), the recalibrated 90% bands cover 0.908, 0.905, and 0.906 in the calm, medium, and volatile terciles—flat to within 0.001—and a per-regime scale does not improve on this. The offset the global scale removes is therefore a benign *constant*, not a conditional failure. A learned latent-space process-noise term (the full RSSM) would still matter for predictive distributions that are non-Gaussian or multi-modal—which the observation-space Gaussian rollout cannot represent—but for the *coverage* of these bands it is not required.

6.9 Derived versus learned: an ensemble Kalman filter on the same field

The paper claims the world model is Bayesian filtering with a learned update rule (§4). We make the claim testable by running a genuine ensemble Kalman filter on the same synthetic field, with the same information: a linear-Gaussian model whose AR coefficient and noise variances are fitted on the training split, an 80-member ensemble, covariance inflation, and perturbed-observation updates. Where the learned core discovers its gate values, the EnKF derives its gain $K = P(P + R)^{-1}$ from covariances.

One implementation detail is worth recording because it decides whether the comparison means anything. Fitting a free 12×12 process covariance from the training residuals yields a nearly diagonal matrix (mean off-diagonal correlation 0.06), so the gain is diagonal, every sensor is filtered in isolation, and an observation informs nobody but itself—a faulty device then cannot contaminate its neighbours, and all fault scenarios return identical numbers. Operational assimilation avoids this by *modelling* the covariance with a spatial correlation function rather than fitting it; substituting a Gaussian kernel with a 1.5 km length scale raises the mean off-diagonal correlation to 0.32 and restores the spatial information flow that makes assimilation assimilation.

The trade is accuracy against calibration. Table 15 gives the comparison. The EnKF matches the linear graph-lite baseline almost exactly at every horizon—unsurprising, since both are linear models of the same field—and it is marginally the better nowcaster, beating the learned core by 5% at $h = 1$. But it falls behind badly as the horizon grows: at $h = 6$ the learned core is 28% better (0.345 against 0.477), because a fitted AR-plus-diffusion transition cannot represent advection the way a learned recurrence can.

The calibration comparison runs the other way, and this is the honest cost of learning. The EnKF’s 90% intervals cover 0.896–0.934 against a nominal 0.90, a mean calibration error of 0.022, *with no tuning at all*: a correct Bayesian posterior is calibrated by construction. The learned rollout’s raw bands cover only 0.71–0.74 (error 0.174) and require the one-parameter rescaling of §6.8 to reach 0.013. Learning

h	test RMSE (norm. units)				90% coverage	
	EnKF	graph-lite	learned core	rollout	EnKF	rollout (raw)
1	0.286	0.281	0.300	0.290	0.896	0.744
2	0.344	0.348	0.319	0.343	0.920	0.711
3	0.389	0.396	0.332	0.369	0.928	0.708
6	0.477	0.480	0.345	0.384	0.934	0.742
calibration error at 90%					0.022	0.174

Table 15: A derived filter against a learned one on the same field. The EnKF tracks the linear baseline and is the better nowcaster, but the learned core is 28% better at six hours. The EnKF is calibrated with no tuning; the learned model’s raw bands are not, and need the rescaling of §6.8 (which reaches 0.013). This is the trade the architecture makes.

buys a large gain in multi-step accuracy and gives up calibration-by-construction in exchange.

Two attempts to close the gap in the filter’s favour. A linear filter beaten by a learned one may simply have been under-specified, so we tried twice to give the EnKF the physics. Neither attempt helped, and both are informative. First we replaced the AR transition with an *advection operator* driven by the known wind, each site inheriting the anomaly from its upwind footprint. The fitted advection speed came out at *zero*: the synthetic plume rotates about a fixed source rather than translating, so a spatial-shift operator has nothing to act on, and the variant reduced to the AR case with an extra fitted parameter (0.499 against 0.477 at $h = 6$). Second we gave the filter the wind directly, conditioning the climatology on wind sector as well as hour-of-day—information the recurrent core was denied and had to infer. This was *worse* at every horizon (0.374 against 0.286 at $h = 1$), because 24×8 conditional buckets overfit the available history. The recurrent core’s advantage therefore does not come from being told more, but from learning a smooth, parameter-efficient representation of a slowly varying hidden regime where a non-parametric conditional mean cannot. We report the 28% margin as earned.

Adaptive noise estimation is not free. The classical counterpart of our trust controller is an adaptive filter that re-estimates each sensor’s observation variance from its own innovations. We implemented it, and it *degraded* performance in every scenario—including the faulted ones it was meant to help (0.326 against 0.290 at $h = 1$ under a stuck sensor)—because the variance estimates are noisy at a 48-hour window and inflate R for healthy sensors along with faulty ones. This is evidence for the stabilizers of §4 rather than against adaptation: the trust controller is not merely adaptive R , but adaptive R with an asymmetric rate, a floor, a hard gate for unambiguous faults, and periodic forced-listening probes. Without that machinery, adapting the observation noise costs more than it returns.

6.10 Late data as free verification

The event-time rule of §5.3 forbids late-arriving records from moving live state or trust. That constraint returns something valuable. While a device was unreachable, every estimate at its location was formed *without* its readings; when the buffer flushes, we hold a prediction and an observation that never informed it. This is an uncontaminated out-of-sample pair at a real location with real ground truth—precisely the virtual-sensor claim, which is otherwise the hardest thing in the system to verify, because unmonitored points have no truth by definition. Outages manufacture it, and an open network with intermittent connectivity manufactures a great deal of it.

Simulating outages on the real Paris series and scoring the trust-weighted inverse-distance estimate at each silent device gives an RMSE of $0.60 \mu\text{g}/\text{m}^3$ with a bias of -0.03 , and predictive intervals that are close to nominal on operationally held-out data ($0.90 \rightarrow 0.94$). That is a verification of the virtual-sensor product against truth it never saw, obtained without holding anything back on purpose.

Trust must weight the target, not only the inputs. A late reading is evidence only if the device was sound. Scoring against devices carrying an injected gain or drift fault reports an RMSE of 4.30 and a 90% coverage of 0.011; the *same* predictions scored against the true field give 0.59. The model appears seven times worse than it is because the yardstick is broken. Since the gate forbids learning trust from

late data, the weight must come from trust as it stood when the device fell silent, together with the device-local physical gates of §6.12, which need no model and apply perfectly well to a stale record.

The sample is not random, and the bias is spatial. Data is usually late because of connectivity, and connectivity is a property of the *site*: signal quality in a basement, behind thick walls, or at the sparse edge of the network is chronically poor rather than occasionally so. The resulting bias is therefore spatial and persistent rather than temporal and transient. Restricting verification to the least-connected sites raises RMSE from 0.52 to 0.67 and drops 90% coverage from 0.935 to 0.890—about a 29% pessimistic skew. The direction is fortunate, since late-data verification understates the model rather than flattering it, and the spatial form is the more tractable one: a chronic per-site property can be measured and stratified, where a weather-driven bias could only be guessed at. Scores should be reported per site group alongside the fleet figure.

Chronic lateness is a coverage gap, not only a verification bias. A device that habitually flushes hours after the fact never contributes to the live cycle. Its own location is therefore served as a virtual estimate *despite being physically instrumented*, and at the densities an early deployment actually reaches, nothing recovers it: in our 16-device Paris fleet no site has a neighbour within 2 km, so removing a device’s own reading leaves its location supported by little more than the regional mean. This exposed an error in the credibility score of §6.11, which counted devices by enrolment. Fleet size must be counted by *timeliness*—the sum of per-device on-time fractions rather than a headcount—or a fleet whose least-connected members rarely arrive in time will advertise support it does not provide. On a plausible connectivity profile, sixteen enrolled devices amount to an effective fleet of 11.7.

6.11 How much should a Plumage user trust an output?

Every relation Murmuration writes is rendered in Plumage beside real measurements, where a forecast from a four-device, one-week deployment looks exactly like one from a mature fleet. Since Canarin’s fleet is open and grows over time, the evidence behind an output varies continuously, and the platform should say so. We therefore measure how output quality actually depends on the two observable properties of a deployment—*fleet size* N and *history length* D —and derive a credibility score from the measured curves rather than asserting a formula.

Two findings shape the score (Table 16). First, the two output families degrade for *different* reasons. Off-device *virtual-sensor* estimates are fleet-limited: leave-one-out reconstruction error falls from 1.03 at $N = 4$ to 0.74 at $N = 24$ and is almost insensitive to history. Forecasts *at* device locations behave oppositely—the linear baseline’s skill is nearly flat in both N and D . Second, and decisively for deployment, the *trained core* is history-hungry in a way the baseline is not: with 12 devices its $h=3$ RMSE is 0.496 at one week—*worse* than graph-lite’s 0.422—then 0.376 at two weeks, 0.336 at one month and 0.317 at three. There is thus a crossover near two weeks below which the heavier model does not deserve to be served at all.

This yields an operational rule rather than a decoration. Each written row carries a **credibility** score in $[0, 1]$ and a band; the score weights history over fleet for forecasts and per-device state (0.7/0.3) and fleet plus *local* density over history for virtual sensors (0.7/0.3, scaled by the number of devices within ~ 1 km, so an extrapolated point is never presented as an interpolated one); and it is multiplied by the mean trust and coverage of the contributing devices, which the platform already tracks. Two hard floors apply: under 14 days of history, or under four devices, the score is capped at 0.20 and the accompanying note instructs the caller to serve the linear baseline and label the output provisional. A deployment of 12 devices with 90 days scores 0.96 for forecasts but only 0.71 for virtual sensors—the honest asymmetry a single global confidence number would hide.

6.12 Multiple channels per device

Canarin devices report many channels at once (Table 2), so we test what joint modelling buys *using only channels a Pico actually reports*: the Plantower’s $\text{PM}_{2.5}$ and PM_{10} , and the BME sensors’ humidity and pressure. We take them over the same 16 greater-Paris locations and 90-day window as §6.6—particulate from the CAMS analysis, humidity and pressure from the meteorological analysis, joined on timestamp exactly as the weather block is joined in deployment. PM_1 , the VOC index and the raw gas signals have no gridded reference product and are not evaluated here.

(a) what limits each output family

	history D (days), at $N = 12$			
$h=3$ RMSE	7	14	30	90
graph-lite	0.422	0.399	0.402	0.397
trained core	0.496	0.376	0.336	0.317

fleet size N

virtual-sensor RMSE	4	6	8	16	24
(leave-one-out)	1.034	0.913	0.807	0.796	0.760

(b) resulting credibility score

fleet	days	forecast	virtual sensor	band / action
4	7	0.10	0.18	low — serve baseline, mark provisional
6	14	0.38	0.37	medium
12	30	0.73	0.63	high
12	90	0.96	0.71	high
24	90	1.00	0.78	high

Table 16: Credibility of model outputs. (a) The trained core needs roughly two weeks of history before it beats the linear baseline, while virtual sensors are limited by fleet size and barely improve with history. (b) The derived score, which the platform writes alongside every row; forecasts and virtual sensors are scored separately because they are limited by different things.

Correlation is the wrong criterion; predictive gain is the right one. Something must decide which channels belong together. Correlation is actively misleading. We measure instead the *predictive gain*: the percentage RMSE reduction in forecasting channel a at $h = 6$ when a few lags of channel b from the same device are added to a ’s own lags—ridge regressions only, no core training. The verdicts nearly invert (Table 17a). $\text{PM}_{2.5}$ and PM_{10} correlate at 0.90 yet the gain is -0.9% : each already predicts itself and the other adds nothing. Humidity correlates with $\text{PM}_{2.5}$ at just 0.09 yet yields the largest gain in the matrix, $+4.9\%$ —the hygroscopic-growth coupling that makes an optical particulate count depend on ambient moisture. Correlation measures co-movement; what a grouping needs is *incremental information*.

Information is not capacity: covariates must condition, not join. The natural next step—put the informative channel into the model—fails, and how it fails is instructive (Table 17b). Adding humidity as an extra *target* costs 13% on $\text{PM}_{2.5}$; adding it as a global *input* only, with heads still restricted to the particulate channels, costs 18%; adding pressure too costs 42%; one shared core over all four costs 26%. Yet the ridge says the information is real. The discrepancy is dimensional: the ridge used three lags of *one device’s own* humidity, whereas the flat core appends one input per device per channel, adding parameters that must be learned from the same limited history. The information is local and per-device; a global vector state cannot express it. This is direct empirical support for the architecture of §4—covariates belong in the per-device conditioning path of the graph encoder, gating how device i ’s own reading is assimilated, rather than in the shared latent—and a limitation of the flat RSSM-lite instantiation used throughout these experiments, which has no per-device structure to condition.

The right way in is multiplicative, not additive. The failure above is not that the information is unusable but that concatenation is the wrong channel for it. Hygroscopic growth is *multiplicative*: an optical counter over-reports by a factor that depends on humidity. Applying the standard growth form as a correction to the observation, $\text{PM}_{\text{dry}} = \text{PM}_{\text{wet}} / (1 + \kappa \text{RH} / (1 - \text{RH}))$ with a single fitted $\kappa = 0.05$, recovers $+4.5\%$ —almost exactly the $+4.9\%$ the ridge said was available—at the cost of *one parameter* rather than one input dimension per device, and it neutralizes the dependence it was meant to remove (correlation between particulate mass and humidity moves from $+0.079$ to -0.043). This is the design’s own claim demonstrated: a covariate belongs in the observation model, correcting the reading, not in the shared latent as another predictor. On real optical hardware the effect should be larger than measured here, since the CAMS product is a model rather than a humidity-sensitive instrument.

<i>(a) correlation vs predictive gain at $h = 6$, real Pico channels</i>		
pair	correlation	predictive gain
$\text{PM}_{2.5} \leftarrow \text{PM}_{10}$	0.90	−0.9%
$\text{PM}_{2.5} \leftarrow \text{humidity}$	0.09	+ 4.9%
$\text{PM}_{10} \leftarrow \text{humidity}$	0.05	+3.8%
$\text{PM}_{2.5} \leftarrow \text{pressure}$	−0.22	−2.2%

<i>(b) cost of adding the informative channel to a flat core</i>		
configuration	$\text{PM}_{2.5}$	PM_{10}
targets alone (per-channel core)	baseline	baseline
+ humidity as an extra target	+13.2%	+10.6%
+ humidity as a global input only	+17.8%	+25.3%
+ humidity and pressure as inputs	+42.0%	+41.7%
all four channels in one shared core	+25.9%	+27.1%

Table 17: Multi-channel results on channels a Pico actually reports. (a) Correlation and predictive gain disagree on every informative pair: humidity is nearly uncorrelated with particulate mass yet is its most useful predictor. (b) Every way of feeding that information into a flat shared state makes matters worse, because the gain is per-device and local while the state is global—which is why covariates condition per-device in the full architecture instead of joining the latent.

How a hard gate crosses a tranche boundary. Two kinds of cross-channel relation must not be conflated. *Statistical* coupling decides tranche membership and is soft, continuous and measured by predictive gain. A *physical* constraint such as $\text{PM}_1 \leq \text{PM}_{2.5} \leq \text{PM}_{10}$ is hard, conclusive and indifferent to the partition—and indeed our own assignment places $\text{PM}_{2.5}$ and PM_{10} in *separate* tranches, so the constraint spans a boundary the partition just created.

That is not a conflict, because gates do not act in the tranche layer at all. They run on the raw record, per device and per timestamp, before any tranche sees the data; needing no model, no neighbour and no history is precisely their value. What a gate cannot do is *attribute*: a violated inequality proves one of the two channels is wrong without saying which, and inflating the fine fraction produces exactly the same violation as deflating the coarse one.

Attribution is where the tranches earn their place. Each channel is scored by the standardized residual against its own expectation, and the channel that broke its own pattern is the culprit; the verdict then reaches every tranche through the shared per-(device, channel) trust weight rather than through any latent state. One detail decides whether this works. An expectation that uses the device’s *own* history absorbs a persistent fault within a step or two, the residual collapses, and attribution falls to chance—measured at 52% for a sustained low-reading fault. Built from neighbours only, the expectation cannot adapt to the device’s own fault, and attribution is exact (100% on both a fine-fraction gain fault and a coarse-fraction low fault, with the gate itself firing on 72–86% of faulted steps and never once in 34,560 clean device-hours). A cross-check must not be self-referential.

Consequences for tranches. Two rules follow, both implemented in the released code. Tranches partition *targets* only: humidity and pressure are covariates and never become tranche members. And within the targets, membership follows measured gain, which on this data leaves $\text{PM}_{2.5}$ and PM_{10} in *separate* tranches despite their 0.90 correlation—the opposite of what correlation-based grouping would do, and consistent with a shared core over the two degrading both.

7 Discussion and Limitations

Runaway trust. The trust loop has a failure mode: a distrusted sensor influences z_t less, so the prediction drifts from it, so the residual grows, so C distrusts it further—a silent amputation delivered with false confidence. The stabilizers in §4 (trust floors, slow η , forced-listening probes) are designed against exactly this, and their calibration on real data is a priority.

Position as a universal excuse. A freely moving Δp can “explain” any fault by teleporting the sensor. Hence position is uncertainty-first, corrections are leashed to the GPS error scale, and larger

moves require human confirmation.

Open participation and its cost. Accepting readings from arbitrary devices maximizes spatial coverage—the whole point of a low-cost network—but widens the provenance problem this paper addresses: an open fleet contains unknown hardware, unknown siting, and unverified locations. The per-device trust, exposure, and position state, together with the device-hierarchy pooling, is precisely the machinery that lets such contributions be admitted without being trusted blindly. Two governance points accompany the technical one: contributions from personal phones carry privacy obligations (RGPD-compliant handling of location and device identifiers), and an open intake invites adversarial or spammy input, so the same trust gate doubles as a first line of abuse resistance.

Scope of evaluation. Our quantitative results are on a synthetic field, on real CAMS $\text{PM}_{2.5}$ for the forecasting comparison, and on live feature extraction—not yet on Canarin’s own Plumage readings, which we cannot access from here, nor on operational data with real faults. The recurrent core’s probabilistic variant emits its own uncertainty but needs a one-parameter recalibration, and on this mostly-observed task its latent collapses; the autoregressive rollout gives the bands a horizon-growing shape but still needs a single global scale; the trust controller is validated on injected faults rather than real device failures. Our other studies are run *per channel*—the single-code case—so the forecasting, calibration, fault and reliability numbers should be read as single-channel results; §6.12 evaluates only the four channels for which a gridded reference exists ($\text{PM}_{2.5}$, PM_{10} , humidity, pressure); PM_1 , the VOC index and the raw gas signals are unevaluated, and its negative results are partly a limitation of the flat core rather than of joint modelling as such. Extending the fault and calibration studies across the real channels—including per-channel trust, the duplicate-sensor gate, and the type-embedding encoder under a genuinely growing code set—remains open. The synthetic study is deliberately constructed to make the baselines informative rather than to flatter a future model, but on-network validation is the essential next step, alongside training the exposure estimator and the reliability controller against real device–weather coupling and real failures.

8 Conclusion

We framed street-level air-quality mapping on a low-cost network as a world-model problem, with the unusual twist that the only available action is epistemic: adjusting the system’s own trust in its sensors. Factoring the problem by update rate makes confounded physical and instrumental effects separable, and a device-type-conditioned observation model handles a genuinely heterogeneous fleet without letting hardware identity leak into the field estimate. A first recurrent core, trained on synthetic data, already clears the strong linear baseline for multi-step forecasting—an advantage that carries to real $\text{PM}_{2.5}$ —and a probabilistic variant emits bands whose calibration we examine; rolling the state forward autoregressively makes those bands compound with horizon and calibrate under a single global scale that we verify holds conditionally as well. Training on operational Plumage readings is the immediate next build; a full latent-space process-noise term would extend the model to non-Gaussian predictive structure, but is not required for the calibration reported here.

Reproducibility

Every number in this paper is produced by a released script, runnable on one CPU with NumPy alone; no autodiff framework, GPU or external service is required except where a public API is named. All gradients are hand-derived and verified against finite differences before use, with the worst relative error reported in the corresponding section.

Result	Script
Baselines, Table 7, Fig. 4	<code>baseline.py</code>
Uncertainty calibration, Table 8	<code>calibration.py</code>
Static context, Table 9	<code>staticcontext.py</code> , <code>thermal.py</code>
Fault injection, Fig. 5	<code>faultinject.py</code>
Hierarchical reliability, Table 10	<code>hierarchical_reliability.py</code>
Recurrent core, Table 11	<code>rssm_lite.py</code>
Real CAMS transfer, Table 12	<code>real_data.py</code>
Probabilistic core, Table 13	<code>rssm_stochastic.py</code>
Autoregressive rollout, Table 14	<code>rssm_rollout.py</code>
Conditional calibration	<code>conditional_calib.py</code>
Late-data verification, §6.10	<code>late_verification.py</code>
Event-time routing	<code>event_time_gate.py</code>
Gate attribution	<code>gate_attribution.py</code>
EnKF comparison, Table 15	<code>enkf.py</code> , <code>enkf_advect.py</code>
Credibility, Table 16	<code>credibility.py</code>
Multi-channel, Table 17	<code>multipollutant.py</code> , <code>tied_core.py</code>
Block split and tranches	<code>blocks.py</code> , <code>type_registry.py</code>
Weather attachment	<code>weather_attach.py</code>
Fleet barometer	<code>fleet_pressure.py</code>
Run everything on an export	<code>run_studies.py -code pm25</code>
Pipeline self-test (no data needed)	<code>run_studies.py -selftest</code>

Data availability. The synthetic field is generated by `baseline.py` from a stated seed. Real particulate, humidity and pressure series come from the CAMS European analysis and the co-located meteorological analysis through a public, key-free API (`real_data.py`, `multipollutant.py`), so every real-data result can be regenerated from scratch. Canarin’s own operational readings are not public; `canarin_data.py` documents the export schema the harness expects.

References

- G. Evensen. The ensemble Kalman filter: theoretical formulation and practical implementation. *Ocean Dynamics*, 53(4):343–367, 2003.
- K. Friston. The free-energy principle: a unified brain theory? *Nature Reviews Neuroscience*, 11(2):127–138, 2010.
- D. Ha and J. Schmidhuber. Recurrent world models facilitate policy evolution. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- D. Hafner, T. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee, and J. Davidson. Learning latent dynamics for planning from pixels. In *ICML*, 2019.
- D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi. Dream to control: learning behaviors by latent imagination. In *ICLR*, 2020.
- T. N. Kipf and M. Welling. Semi-supervised classification with graph convolutional networks. In *ICLR*, 2017.
- A. Lewis and P. Edwards. Validate personal air-pollution sensors. *Nature*, 535:29–31, 2016.
- Y. Li, R. Yu, C. Shahabi, and Y. Liu. Diffusion convolutional recurrent neural network: data-driven traffic forecasting. In *ICLR*, 2018.
- B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng. NeRF: representing scenes as neural radiance fields for view synthesis. In *ECCV*, 2020.
- T. R. Oke. Street design and urban canopy layer climate. *Energy and Buildings*, 11(1–3):103–113, 1988.
- V. Sitzmann, J. N. P. Martel, A. W. Bergman, D. B. Lindell, and G. Wetzstein. Implicit neural representations with periodic activation functions. In *NeurIPS*, 2020.
- Y. Zheng, F. Liu, and H.-P. Hsieh. U-Air: when urban air quality inference meets big data. In *ACM SIGKDD*, 2013.